Test

- Entry test (6 out of 10 Tasks)
- Further Questions
- True or False
- Calculation (Synchronizing deadlock, paging algorithms)

Operating System Entry Test

Questions

1. What is the **Kernel (definition)**?
2. What is IEEE POSIX?
3. What is a task?
4. Name at least three preemptive schedulers!
5. List the most important three states of a task
6. What is the purpose of aging in scheduling?
7. What is the Swap?
8. What is the meaning of a page fault error (interrupt)?
9. What is synchronization (definition)?
10. What kinds of data are stored in an inode?

Answers

1. Protected or system core that boots the program. Manages the user processes
2. Set of standards, how to create the interfaces for the OS and defines the APIs, how to control the processes, manages them, how file operations, directory operations should look like. API definition. It is a standard operating system and environment, including a command interpreter and common utility programs.
3. Program during execution
4. Round Robin, Shortest Remaining Time First (SRTF), Multilevel Feedback Queue
5. Ready to Run, Run, Waiting
6. Avoid starvation, lower priority jobs are prone for starvation due to not getting resources.
7. Part of background storage/disk, physical operating memory is extended. If no enough memory, then it is put on physical memory and then swap back
8. Whenever program access variable that is not on the working memory, it makes page fault error, and the handler of the exception, the corresponding page will be fetched into the memory and it will resume running
9. Coordination among tasks by means of constraining operation in time

10. Metadata of a file or a directory of the operating systems. Timestamp, session date, file type, etc... Permissions (9 file character combinations). INode belongs to a file, and that is the metadata, what do we need? → Where the files is

---

True or False

(T) (F)
Warning: Negative points for the block

Operating system basics, structure and operation

1. The source of an operation system may contain more than ten million lines of code (T)
2. The kernel runs in protected mode and governs applications operating in user mode (T)
3. In embedded OS the application and the OS are not always separated, sometimes they are compiled into a single executable (T)
4. Systemd is a sysinit alternative (T) → sysinit read config. files and what services should be started at the boot process. systemd is pratically a better version of sysinit, but a complete different implements but does the same and more organized
5. A modular kernel may not be a monolithic program (F)

Task Management

1. Response time is the time interval between task creation and its first output (response) (T)
2. The ps command on Unix systems lists files (F)
3. Memory-intensive tasks may become I/O-intensive if there is not enough memory (T) → Because of swapping, swap space, I/O operations
4. A process is a sequentially executed thread (F) → thread is an atomic unit, process can have multiple threads
5. All administrative data of a task is stored in the kernel's memory (F) → rather false, e.g. open files are stored in the process memory area. Kernel only stores the most important, and used by the management of the lifecycle

Scheduling

1. It is very hard to implement the shortest job first (SJF) scheduler in real operating system (T)
2. Schedules schedule process not threads (F)
3. The Round-Robin scheduling avoids starvation (T)
4. The SJF may be seen as a priority scheduler (T)
5. Today's operating systems mainly use the hard CPU affinity in multiprocessor scheduling (F)

Memory management

1. Tasks may not use more memory that is physically available in the system (F) → we can using swapping, virtual memory, etc...
2. The number of page tables are significantly higher than the number of frame tables (T) → Typically there is as much page tables as there are processes running. Typically there is only one frame table
3. Old page fault frequency is higher, the memory-intensive tasks spend more time in blocked (waiting) state (T)
4. The page fault frequency always decreases if we add more physical memory to the system (F)
5. Today's operating systems mainly use anticipatory paging because it is able to predict what pages will be used in the near future (F)

Short Answers

Questions

1. List a few examples for heterogeneous systems. (2p)
2. The message queue is a direct/indirect communication method. (underline the appropriate) (1p)
3. The main parts of a tasks runtime context are ... (2p)
4. Main frame table fields are ... (2p)
5. What is the purpose of the copy-on-write technique? (2p)
6. What is the main difference between the clock and the second chance page replacement algorithms? (1p)
7. Write down the names of two different command line shells (1p)
8. What is the role of the rpcgen command in remote procedure call implementations?
9. Write down a small code fragment that illustrates how semaphore operations can protect a critical section?
10. What is more reliable: RAID0 or RAID1?

Answers

1. Heterogeneous system are computing environments that integrate different types of processing units, such as CPUs, GPUS, and FPGAs to perform tasks more efficiently.
2. Indirect
3. State on the CPU (values of registers), general purpose register/computational registers, Program Counter, Status Registers (multiple flags), page tables, pointers (stack pointer), description and properties of the process itself, status information, I/O operations, open files

4. Status bit, dirty bit, valid bit, the number of the page table. Further bits e.g. Log bit (logged or not)
5. Related to fork() → duplicates the current running process. Copy and recreate all the data of that process. All the memory should be copied, but is ineffective. Instead of copying everything, only the page tables are copied, we end up with two identical processes and contents, which point to the same memory frames, in a shared manner. Contents should belong to one of the processes.
6. Clock algorithm uses pointers (point to object on the queue) ?????
7. Bash, Windows PowerShell
8. Generates C Code to implement a RPC (Remote Procedure Call) protocol. It will generate all of this code that the developers only implement the body of the functions on the server side and all the handling of the returning on the client side
9. ```
   int sem;
   P(sem) sem_wait
   .
   .
   .
   V(sem) sem_release
   ```
10. **RAID1**
   ```